

UNIT 1

RELATIONAL DATABASES

Purpose of Database System – Views of data – Data Models – Database System
Architecture – Introduction to relational databases – Relational Model – Keys –
Relational Algebra – SQL fundamentals – Advanced SQL features – Embedded
SQL– Dynamic SQL

UNIT - 1

INTRODUCTION TO DBMS

INTRODUCTION

Data

Data is a raw material. It can be any [character](#), word, number, date etc.

Example: 5, 27-11-2018, Nagercoil, Sherill

Database

Database is a collection of inter-related data which is used to retrieve, insert and delete the data efficiently. For example, consider employee database consists of following columns- eno, ename and salary.

The diagram shows a table with three columns: 'Eno', 'ename', and 'salary'. The rows contain data for three employees: (1001, Anand, 5000), (1002, Prajan, 6000), and (1003, Kumar, 7000). Annotations with arrows point to the table name 'Employee', the columns, and the rows.

Eno	ename	salary
1001	Anand	5000
1002	Prajan	6000
1003	Kumar	7000

Fig. 1.1 Example of Database

DBMS

A **database-management system (DBMS)** is a collection of interrelated data and a set of programs to access those data.

The goal of a DBMS is to provide a way to store and retrieve database information that is both convenient and efficient.

Example:

MySQL, Oracle, SQL Server, IBM DB2, PostgreSQL

PURPOSE OF DATABASE SYSTEMS

The typical file-processing system is supported by a conventional operating system. The system stores permanent records in various files, and it needs different application programs to extract records from, and add records to, the appropriate files. Before database management systems (DBMS) came along, organizations usually stored information in such systems.

Disadvantages of File Systems Over Database Systems

1. Data Redundancy:

Data redundancy and inconsistency. Since different programmers create the files and application programs over a long period, the same information may be duplicated in different files. This leads to data redundancy.

Example: A student is having record in college office and in computer department. Then his , telephone number, address will be same in both the departments. This redundancy leads to higher storage and access cost.

In addition, it may lead to **data inconsistency**, that is, the various copies of the same data may no longer agree.

Example : A changed student address may be reflected in the computer department records but not in the college office.

2. Difficulty in Accessing Data: Accessing data is not convenient and efficient in file conventional processing system. More responsive data-retrieval systems are required for general use.

Example: one of the bank officers needs to find out the names of all customers who live within a particular postal- code area. If there is no application program for this means, the officer has 2 alternatives: 1. Preparing the list manually from the list of all customers. 2. Ask system programmer to write the necessary application programs.

3. .Data isolation. Because data are scattered in various files, and files may be in different formats, writing new application programs to retrieve the appropriate data is difficult.

4. Integrity Problems: The data values stored in the database must satisfy certain types of **consistency constraints**.

Example: Suppose the university maintains an account for each department, and records the balance amount in each account. Suppose also that the university requires that the account balance of a department may never fall below zero. However, when new constraints are added, it is difficult to change the programs to enforce them.

6. **Atomicity Problems:** Any operation on database must be atomic, (i.e.) it must happen in its entirety or not at all.

Example: If you are buying a ticket from railway and you are in the process of money transaction. Suddenly, your internet got disconnected then you may or may not have paid for the ticket. If you have paid, then your ticket will be booked and if not then you will not be charged anything. That is called consistent state, means you have paid or not.

7. **Concurrent Access Anomalies:** Multiple users are allowed to access data simultaneously, this is for the sake of better performance and faster response.

Example: Consider an operation to debit (withdrawal) an account. The program reads the old balance, calculates the new balance, and writes new balance back to database. Suppose an account has a balance of Rs. 5000. Now, a concurrent withdrawal of Rs. 1000 and Rs. 2000 may leave the balance Rs. 4000 or Rs. 3000 depending upon their completion time rather than the correct value of Rs. 2000.

8. **Security Problems:** Database should be accessible to users in limited way. Not every user of the database system should be able to access all the data.

Example : In a university, payroll personnel need to see only that part of the database that has financial information. They do not need access to information about academic records. To overcome these difficulties, the database management system (DBMS) was developed.

VIEWS OF DATA

Data Abstraction

Database systems are made-up of complex data structures. To ease the user interaction with database, the developers hide internal irrelevant details from users. This process of hiding irrelevant details from user is called data abstraction.

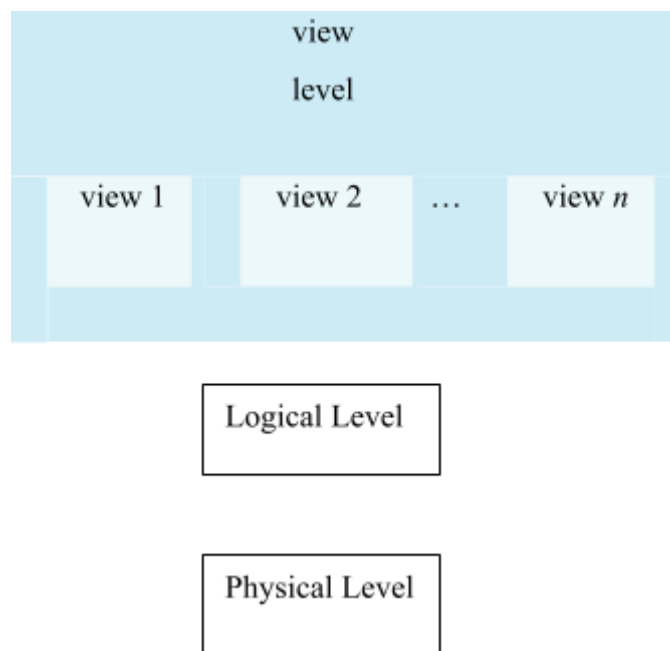


Fig. 1.2 Views of Data

Physical level (Internal Level):

- ❖ This is the lowest level of data abstraction.
- ❖ It describes *how* the data are actually stored.
- ❖ The physical level describes complex low-level data structures in detail.

Logical level (Conceptual Level):

- ❖ This is the middle level of data abstraction.
- ❖ The logical schema describes the structure of the whole database.
- ❖ The logical level describes what data are to be stored in the database and also describes what relationship exists among those data.

- ❖ Database administrators, who must decide what information to keep in the database, use the logical level of abstraction.

View level (External Level):

- ❖ This is the highest level of data abstraction.
- ❖ It describes only part of the entire database.
- ❖ The view level of abstraction exists to simplify their interaction with the system
- ❖ At the external level, a database contains several schemas that sometimes called as subschema. The subschema is used to describe the different view of the database.

Many high-level programming languages support the notion of a structured type. For example, we may describe a record as follows:

type *instructor* = record

ID : **char** (5);

name : **char** (20);

dept name : **char** (20);

salary : **numeric** (8,2);

end;

This code defines a new record type called *instructor* with four fields. Each field has a name and a type associated with it. A university organization may have several such record types, including

department, with fields *dept name*, *building*, and *budget*

course, with fields *course id*, *title*, *dept name*, and *credits*

student, with fields *ID*, *name*, *dept name*, and *tot cred*

At the physical level, an *instructor*, *department*, or *student* record can be described as a block of consecutive storage locations. The compiler hides this level of detail from programmers.

At the logical level, each such record is described by a type definition, as in the previous code segment, and the interrelationship of these record types is defined as well. Programmers using a programming language work at this level of abstraction. Similarly, database administrators usually work at this level of abstraction.

At the view level, several views of the database are defined, and a database user sees some or all of these views. In addition to hiding details of the logical level of the database, the views also provide a security mechanism to prevent users from accessing certain parts of the database.

SCHEMA AND INSTANCES

Instance:

The collection of information stored in the database at a particular moment is called an **instance** of the database. Database schema defines the variable declarations in tables that belong to a particular database; the value of these variables at a moment of time is called the instance of that database.

Schema:

The overall design of the database is called the database schema.. Schema is of three types: Physical schema, logical schema and view schema.

Application programs are said to exhibit **physical data independence** if they do not depend on the physical schema, and thus need not be rewritten if the physical schema changes.

DATA MODELS

Introduction

Data Model is a collection of conceptual tools for describing data, data relationships, data semantics and consistency constraints.

Data Models can be classified into three categories

1. Object-based logical model
2. Record-based logical model

Object-Based Logical Models

It is a collection of conceptual tools for describing data, data relationships and data constraints. There are various object-based models like,

- a. Entity-Relationship Model
- b. Object Oriented Model

a. Entity-Relationship Model

- ❖ The entity-relationship (E-R) data model uses a collection of basic objects, called *entities*, and *relationships* among these objects.
- ❖ An entity is a “thing” or “object” in the real world that is distinguishable from other objects. The entity-relationship model is widely used in database design.

b. Object-Oriented Data Model

- ❖ Object-oriented data model that can be seen as extending the E-R model with notions of encapsulation, methods , and object identity.
- ❖ The object-relational data model combines features of the object-oriented data model and relational data model.

Record Based Logical Models

The record based model describes the data structures and access techniques of a DBMS. There are three types of record-based models viz.

- a) Relational Model
- b) Hierarchical Model
- c) Network Model

a) Relational Model.

- ❖ The relational model uses a collection of tables to represent both data and the relationships among those data.
- ❖ Each table has multiple columns, and each column has a unique name.
- ❖ Tables are also known as **relations**.
- ❖ The relational model is an example of a record-based model.

- ❖ Record-based models are so named because the database is structured in fixed-format records of several types.
- ❖ Each table contains records of a particular type.
- ❖ Each record type defines a fixed number of fields, or attributes.
- ❖ The columns of the table correspond to the attributes of the record type.

b) Hierarchical Data Model

In Hierarchical data model,

- ❖ Organizes the data in a Tree Structure
- ❖ Hierarchy of parents and child segments
- ❖ Data is represented by a collection of record types

c) Network Data Model

In Network data model,

- ❖ Organizes the data in a Graph Structure
- ❖ Relationships among data are represented by links
- ❖ Data is represented by a collection of record types

DATABASE SYSTEM ARCHITECTURE

Two Tier and Three Tier Architecture

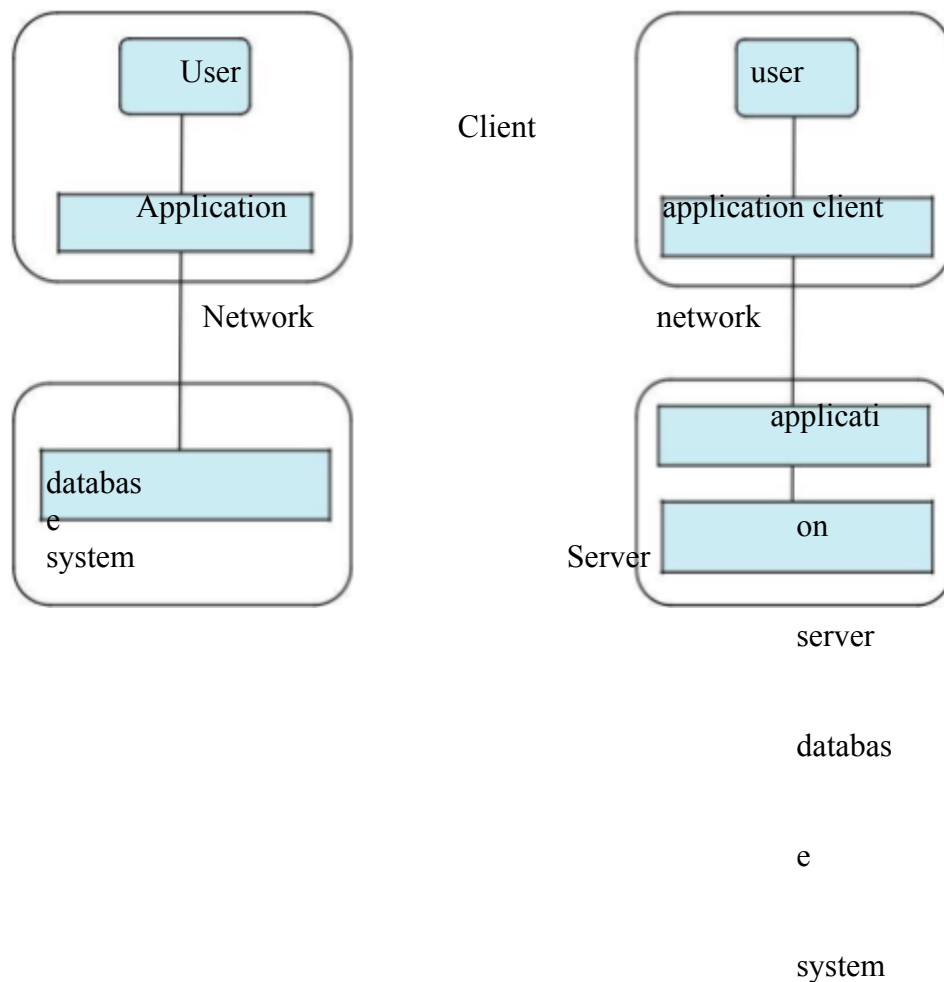


Fig.1.3.(a) Two-tier
architecture

(b) Three-tier architecture

The architecture of a database system is greatly influenced by the underlying computer system on which the database system runs. Database systems can be centralized, or client-server, where one server machine executes work on behalf of multiple client machines. Database systems can also be designed to exploit parallel computer architectures. Distributed databases span multiple geographically separated machines.

Database applications are usually partitioned into two or three parts, as in Figure 1.1. In a **two-tier architecture**, the application resides at the client machine, where it invokes database system functionality at the server machine through query language statements. Application program interface standards like ODBC and JDBC are used for interaction between the client and the server.

In contrast, in a **three-tier architecture**, the client machine acts as merely a front end and does not contain any direct database calls. Instead, the client end communicates with an **application server**, usually through a forms interface.

The application server in turn communicates with a database system to access data. The **business logic** of the application, which says what actions to carry out under what conditions, is embedded in the application server, instead of being distributed across multiple clients. Three-tier applications are more appropriate for large applications, and for applications that run on the World Wide Web.

Components of DBMS

The functional components of a database system can be divided into 3 units:

- a) Storage Manager**
- b) Query Processor**
- c) Database Users and Administrators**

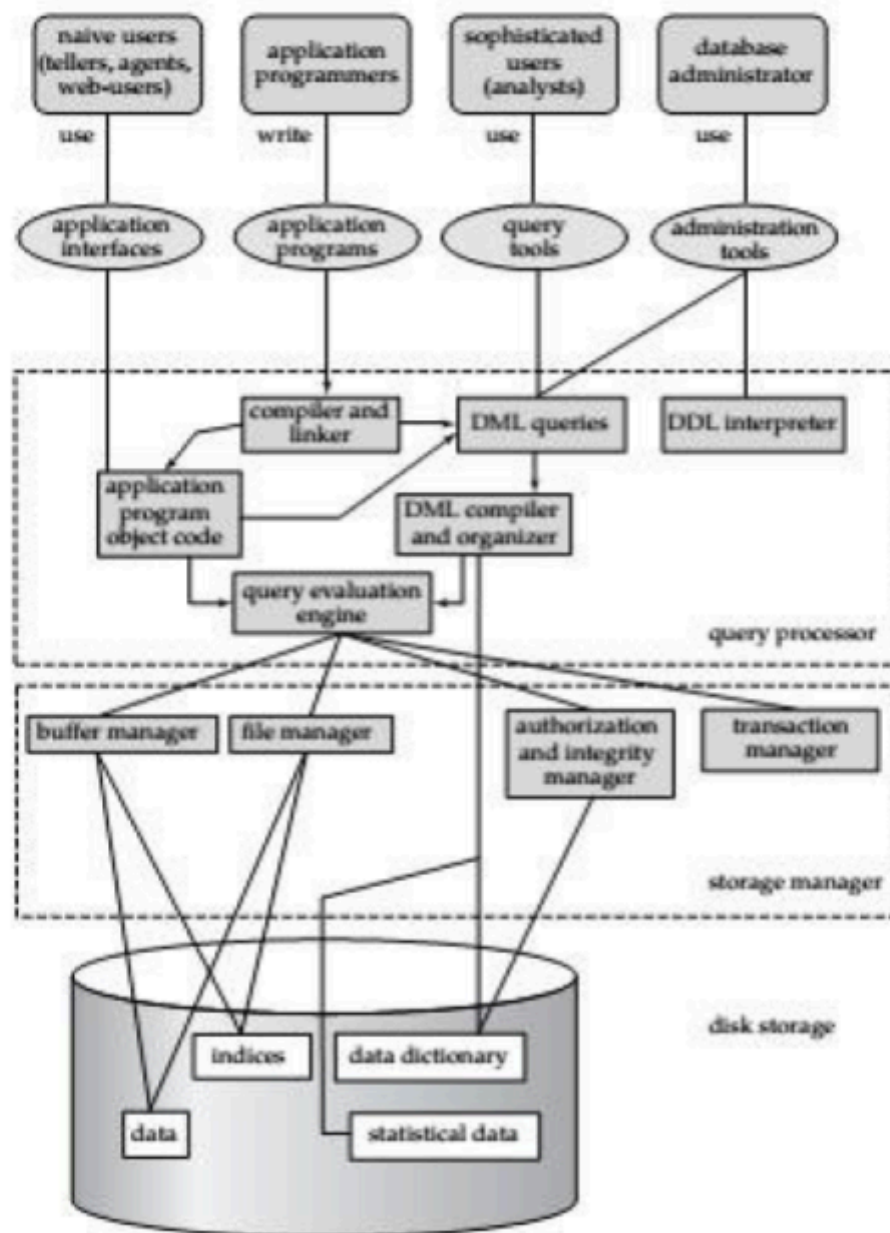


Figure 1.4 System structure.

a) **Storage Manager:** It is important because database typically requires a large amount of storage space. It is a program module that provides an interface between the low level data stored in the database and the application programs and queries submitted to the system.

The storage manager translates the various DML statements into low level file system commands. Thus the storage manager is responsible for storing, retrieving and updating the data in the database.

It includes the following components.

- (1) **Authorization and Integrity Manager:** which tests for the satisfaction of integrity constraints and checks the authority of users to access data.
- (2) **Transaction Manager:** which ensures that the database remains in a consistent state when the system failures and that concurrent transaction executions proceed without conflicting.
- (3) **File Manager:** which manages the allocation of storage space on disk and data structures used to store that information's.
- (4) **Buffer Manager:** which is responsible for fetching data from disk into main memory.

Data Structure used by the Storage Manager

- ❖ **Data files**, which stores database itself.
- ❖ **Data dictionary**, which stores meta data about the structure of the database, in particular schema of the database.
- ❖ **Indices**, which can provide fast access to data items. Hashing technique is used for accessing.

b) Query Processor

It contains the following components.

- **DDL Interpreter:** which interprets DDL statements (i.e., converts DDL statement into low level data).
- ❖ **DML Compiler:** Which translates DML statements into an low level instructions that the query evaluation engine understands. Also performs Query optimization.
- ❖ **Query Evaluation Engine:** Which executes low level instructions generated by the DML compiler.

c) Database Users

❖ **Application programmers:** Computer professionals who interact with the system through DML calls, which are embedded in a program written in a host language (Cobol, PL/I, Pascal, C).

❖ **Sophisticated users:** Interact with the system through the database query language.

❖ **Specialized users:** Specialized users who write specialized database applications that do the into the traditional data processing frame work (e.g: CAD system, knowledge based and expert systems.)

❖ **Naive Users:** Interact with the system by invoking one of the permanent applications.

❖ **Database Administrator:** A person who has such central control over the system is called a database administrator (DBA).

The functions of a DBA include:

1. **Schema definition:** The DBA creates the original database schema by executing a set of data definition statements in the DDL.
2. **Storage structure and access-method definition.**
3. **Schema and physical-organization modification:** The DBA carries out changes to the schema and physical organization to reflect the changing needs of the organization to improve performance.
4. **Granting of authorization for data access:** By granting different types of authorization, the database administrator can regulate which parts of the database various users can access. The authorization information is kept in a special system structure that the database system consults whenever someone attempts to access the data in the system.
5. **Routine maintenance:** Examples of the database administrator's routine maintenance activities are:
 - ✓ Periodically backing up the database

- ✓ Ensuring that enough free disk space is available for normal operation
- ✓ Monitoring jobs running on the database

1.6.

RELATIONAL MODEL

A relational database management system (RDBMS) is a database management system based on relational model introduced by E.F.Codd. In relational model, data is represented in terms of *tuples (rows)*.

In relational database, a table is a collection of data elements organized in terms of rows and columns. A table is also considered as convenient representation of relations. But a table can have duplicate tuples while a true relation cannot have duplicate tuples. Table is the simplest form of data storage. Below is an example of *employee table*.

Emp id	Name	Age	Salary
1	Baskar	34	13000
2	Dharshan	28	15000
3	Prajan	20	18000
4	Pradeep	42	19020

A single entry in a table is called a *record* or *row*. A record in a table represents set of related data. For example, the above Employee table has 4 record. Following is an example of single record.

1	Baskar	34	13000
---	--------	----	-------

A table consists of several records (row); each record can be broken into several smaller entities known as *fields*. The above employee table consists of four fields Emp_id, Name, Age and Salary.

In relational table, a column is a set of value of a particular type. The term *attribute* is also used to represent a column. For example, in Employee table, Name is a column that represent names of employee.

Name
Baskar
Dharshan
Prajan
Pradeep

E. F. Codd's Rule

These rules were defined by E.F. Codd's in a paper published in 1985. They specify what a relational database must support in order to be relational.

Rule 1: The information Rule

"All information in a relational data base is represented explicitly at the logical level and in exactly one way - by values in tables."

- ☞ All information including table names, column names and column data types should be available in some table with in the database.
- ☞ Tables that hold such information constitute the data dictionary.
- ☞ Data are represented only one way: as values within columns within rows.
- ☞ Simple, consistent and versatile.
- ☞ The basic requirement of the relational model.

Rule 2: Guaranteed access Rule

"Each and every datum (atomic value) in a relational data base is guaranteed to be logically accessible by resorting to a combination of table name, primary key value and column name."

- ☞ Every value can be accessed by providing table name, column name and key.
- ☞ All data are uniquely identified and accessible via this identity.
- ☞ Most RDBMS do not make the definition of the primary key mandatory and are deficient to that extent

Rule 3: Systematic treatment of null values

"Null values (distinct from the empty character string or a string of blank characters and distinct from zero or any other number) are supported in fully relational DBMS for representing missing information and inapplicable information in a systematic way, independent of data type."

- ☞ Separate handling of missing and/or non applicable data.
- ☞ This is distinct to zero or empty strings

Rule 4: Dynamic on-line catalog based on the relational model

"The data base description is represented at the logical level in the same way as-ordinary data, so that authorized users can apply the same relational language to its interrogation as they apply to the regular data."

- ☞ Catalog (data dictionary) can be queried by authorized users as part of the database.
- ☞ The catalog is part of the database.

Rule 5: Comprehensive data sub-language Rule

A relational system may support several languages and various modes of terminal use (for example, the fill-in-the-blanks mode). However, there must be at least one language whose statements are expressible, per some well-defined syntax, as character strings and that is comprehensive in supporting all the following items

- ☞ Data Definition
- ☞ View Definition
- ☞ Data Manipulation (Interactive and by program).
- ☞ Integrity Constraints

Rule 6: View updating Rule

All views that are theoretically updatable are also updatable by the system.

Rule 7: High-level insert, update and delete

The capability of handling a base relation or a derived relation as a single operand applies not only to the retrieval of data but also to the insertion, update and deletion of data.

Rule 8: Physical data independence

"Application programs and terminal activities remain logically unimpaired whenever any changes are made in either storage representations or access methods."

Rule 9: Logical data independence

"Application programs and terminal activities remain logically unimpaired when information-preserving changes of any kind that theoretically permit un-impairment are made to the base tables."

Changes to the logical level (tables, columns, rows, and so on) must not require a change to an application based on the structure. Logical data independence is more difficult to achieve than physical data independence.

Rule 10: Integrity independence

"Integrity constraints specific to a particular relational data base must be definable in the relational data sub-language and storable in the catalog, not in the application programs."

- ☐ Integrity constraints are to be stored in the catalog not the programs.
- ☐ Alterations to integrity constraints should not affect application programs.
- ☐ This simplifies the programs.
- ☐ It is not always possible to do this.

Rule 11: Distribution independence

The data manipulation sub language of an RDBMS must enable application programs and queries to remain logically unchanged whether & whenever data is physically centralized or distributed.

Rule 12: The Non-Subversion rule

If the system provides a low-level (record-at-a-time) interface, then that interface cannot be used to subvert the system, for example, bypassing a relational security or integrity constraint.

All the database access must be controlled through the DBMS so that the integrity of the database cannot be compromised without the knowledge of the user.

KEYS

A key allows us to identify a set of attributes and thus distinguishes entities from each other. Keys also help to uniquely identify relationships, and thus distinguish relationships from each other.

Different types of keys are:

- 📖 Super key
- 📖 Candidate key
- 📖 Primary key
- 📖 Foreign key

1. **Super key:** A **superkey** is a set of one or more attributes that, taken collectively, allow us to identify uniquely a tuple in the relation. For example, consider the student relation.

Student (Rollno, Name, Age) is a *Super key*.

2. **Candidate key:** A table which have more than one attribute that uniquely identify an instance of an entity set. These attributes are called *Candidate keys*.

For example, Consider the car relation,

Car (license_no, engine_serial_no, make, model, year) In this relation, we can find the two *Candidate keys*:

license_no and engine_serial_no.

3. **Primary key:** An attribute which is unique and not null, can identify an instance of the entity set is termed as *Primary key*.

For example, Consider the employee relation,

Employee (eno, ename, sex, doj, dob, sal, job, dno) in this eno is the *Primary key*.

4. **Foreign key:** An attribute in one relation whose value matches the primary key in some other relation is called a *Foreign key*.

For example, Consider the two relations dept and employee,

Dept (dno, dname, dloc)

Employee (eno, ename, sex, doj, dob, sal, job, dno)

In the above relations, for dept relation dno is the *primary key*, for employee relation eno is the *primary key* and here we can find that the employee relation - dno matches with the dept relation - dno, so that employee relation dno is known as *Foreign key*.

RELATIONAL ALGEBRA

1.6.1 Introduction

Query language is a language in which user requests information from the database.

Categories

- (i) Procedural - Eg., - Relational algebra.
- (ii) Non-procedural - Eg., - Relational calculus.

- (i) **Procedural language:** The user instructs the system to perform a sequence of operations on the database to compute the desired result.

Example: Relational algebra.

- (ii) **Non-procedural language:** The user describes the desired information without giving a specific procedure for obtaining that information.

Example: Relational calculus.

Relational Algebra

- Procedural language.
- It consists of set of operations that take one or more relations as input and produces new relation as output.
- The operations can be divided into
 - ❖ **Basic operations** Select, project, union, rename, set difference & cartesian product.
 - ❖ **Additional operations** that can be expressed in terms of basic operations □ Set intersection, natural join, division and assignment.
 - ❖ **Extended operations** □ Aggregate operations and outer join.

Basic Operations

(a) The select operation (□)

It is used to select a subset of the tuples from a relation that satisfy a selection condition.

Syntax

□ <Selection Condition> (R)

Example

Select those tuples of the loan relation where the branch is “Perriridge”.

□ branch-name = “Perriridge”

(loan) The results of □ branch-name = “Perriridge”

(loan)

loan-number	branch-name	amount
L-15	Perryridge	1500
L-16	Perryridge	1300

(b) The Project Operation (π)

The Project operation, on the otherhand, selects certain columns from the table and discards the other columns. If we are interested in only certain attributes of a relation, we use the Project operation to project the relation over these attributes only.

Syntax

$\pi_{\langle \text{Attribute list} \rangle} (R)$

Example

List the loan-no,amount from the loan relation.

$\pi_{\text{loan-no, amount}} (\text{loan})$

The results of $\pi_{\text{loan-no, amount}} (\text{loan})$

loan-number	amount
L-11	900
L-14	1500
L-15	1500
L-16	1300
L-17	1000
L-23	2000
L-93	500

Composition of relational operations - is the combination of select and project operation.

Example

Find those customers who live in “Rye”

$\pi_{\text{customer-name}} (\sigma_{\text{customer-city} = \text{“Rye”}} (\text{customer}))$

The results of $\pi_{\text{customer-name}} (\sigma_{\text{customer-city} = \text{“Rye”}} (\text{customer}))$

customer-name
Curry
Smith

(c) The Union Operation ($\cup$)

The result of this operation, denoted by $R \cup S$, is a relation that includes all tuples that are either in R or in S or in both R and S. Duplicate tuples are eliminated.

Synta
x

$R \sqcup S$

Example

Find the names of customers who have either an account or a loan or both.

Names of all customers with a loan is

$\pi_{\text{customer-name}}(\text{borrower})$

Names of all customers with account is

$\pi_{\text{customer-name}}(\text{depositor})$

So the expression needed is $\pi_{\text{customer-name}}(\text{borrower}) \cup \pi_{\text{customer-name}}(\text{depositor})$ The

result of $\pi_{\text{customer-name}}(\text{borrower}) \cup \pi_{\text{customer-name}}(\text{depositor})$

customer-name
Adams
Curry
Hayes
Johnson
Jones
Lindsay
Smith
Turner
Williams

(d) The set difference operation ()

It is used to find tuples that are in one relation but are not in another.

Syntax

$R - S$

Example

Find all customers who have an account but not have a

loan. $\pi_{\text{customer-name}}(\text{depositor}) - \pi_{\text{customer-name}}(\text{borrower})$

The result of $\pi_{\text{customer-name}}(\text{depositor}) - \pi_{\text{customer-name}}(\text{borrower})$

customer-name
Johnson
Lindsay

(e) The Cartesian Product Operation (X)

It allows us to combine information from any 2 relations.

Defined as

$$r \times s = \{ tq \mid t \in r \text{ and } q \in s \}$$

Assume that attributes of $r(R)$ and $s(S)$ are disjoint.

For Example

			Relation r		Relation s		$r \times s$	
			A	B				
C	D	E		1				
				2				
	10	a						
		a						
	20	b						
	10	b						

A	B	C	D	E
1			10	a
1			10	a
1			20	b
1			10	b
2			10	a
2			10	a
2			20	b
2			10	b

(f) The Rename operation: (rho-)

- ❖ Allow us to name the relational algebra expressions.
- ❖ Allow us to refer to a relation by more than one name.

Example: $\rho_X(E)$ returns the expression E under the name X.

If a relational algebra expression E has arity n , then

$$\rho_X(A_1, A_2, A_3, \dots, A_n) (E)$$

returns the result of expression E under the name X, and with the attributes renamed to $A_1, A_2, A_3, \dots, A_n$.

For example, consider the Book relation with attributes Title, Author, Year and Price. The rename operator is used on Book relation as follows.

$$\rho_{\text{Temp}}(\text{Bname}, \text{Aname}, \text{Pyear}, \text{Bprice}) (\text{Book})$$

Here both the relation name and the attribute names are

renamed.

$\rho_{Temp}^{(Book)}$ – Here only the relation name is renamed.

Additional Operations

(g) Set intersection operation ($\cap$)

- o Notation used is $r \cap s$.

- o It is defined as $r \bowtie s = \{ t / t \in r \text{ and } t \in s \}$.
- o Assume r, s have the same attributes.
- o The attributes of r and s are compatible.
- o $r \bowtie s = r - (r - s)$.

Example

Consider relation r, s

r		s		$r \bowtie s$	
A	B				
1	2	1	2	1	2
1	2	1	2	1	2
1	1				

A	B
	2
	3

A	B
1	2

(h) Natural Join operation ()

s is a relation

➤ Notation used is (r s)

- Let r and s be relations on scheme R and S respectively. Then r



on schema RUS obtained as follows

- Consider each pair of tuples t_r from r and t_s from s .
- If t_r and t_s have the same value on each of the attributes in R and S, add a tuple t to the result, where
 - ◆ t has the same value as t_r on r .
 - ◆ t has the same value as t_s on s .

Example

$R = (A, B, C, D)$

$S = (E, B, D)$

Result schema = $(A, B, C, D,$

$E)$ is defined as

$r \bowtie s \iff \text{RUS} \llbracket r.A1 \parallel s.A1 \parallel r.A2 \parallel s.A2 \parallel \dots \parallel r.An \parallel s.An \rrbracket$

r

s

A	B	C	D
	1		a
	2		a
	4		b
	1		a
	2		b

⋈

B	D	E
1	a	
3	a	
1	a	
2	b	
3	b	

⋈

⋈

$r \bowtie s$

A	B	C	D	E
	1		a	
	1		a	
	1		a	
	1		a	
	2		b	

(i) Division operation ($r \div s$)

- Suited to the queries that includes the phrase “for all”.
- Let r and s be relations on schema R and S respectively

where $R = (A_1, A_2, A_3, \dots, A_m, B_1, B_2, B_3, \dots, B_n)$

$S = (B_1, B_2, \dots, B_n)$

The results of rs is a relation on

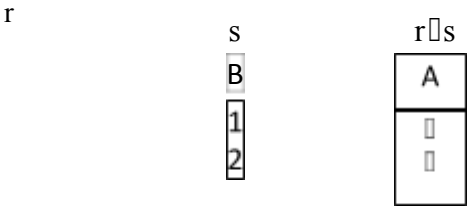
schema $R - S = (A_1, A_2, \dots, A_m)$

$r \div s = \{t / t \ll_{R-S(t)} \ll u \in S(tu \in r)\}$

For Example:

Relation r, s

A	B
1	1
1	2
1	3
1	1
1	1
1	1
1	3
1	4
1	6
1	1
1	2



(j) Assignment Operation ($\leftarrow$)

- The assignment operation ($\leftarrow$) provides a convenient way to express complex queries.
- Write query as a sequential program consisting of a series of assignments followed by an expression where value is displayed as a result of the query.
- Assignment must always be made to a temporary relation variable.

Example: Write $r \div s$ as

```
temp ← 1 ← R ⋈ S ⋈ r
temp ← 2 ← R ⋈ S ⋈ temp1 S ⋈ r
result = temp 1 – temp 2
```

- The result to the right of the $\leftarrow$ is assigned to the relation variable on the left of the $\leftarrow$.
- May use variables in subsequent expressions.

Extended Relational-Algebra Operations

- Generalized projection
- Outer join

□ Aggregate functions.

(a) Generalized Projection

- Extends the projection operation by allowing arithmetic functions to be used in the projection list.

□_{F₁, F₂ ..., F_n}(E). E is any Relational algebra expression.

- Each of F₁, F₂, ..., F_n are arithmetic expressions involving constants and attributes in the schema of E.

- Given relation credit (customer-name, limit, credit - balance) find how much more each person can spend:

$\pi_{\text{customer_name}, \text{limit_credit_balance}}(\text{credit})$

(b) Aggregate Functions and Operations

- Aggregate function takes a collection of values and returns a single value as a result.

Avg., min., max., sum, count are few aggregate functions.

- Aggregate operations in Relational algebra is defined as $G_1, G_2, \dots, G_n F_1(A_1), F_2(A_2), \dots, F_n(A_n)(E)$.

Here E is any relational algebra expression.

$G_1, G_2, \dots, G_n$ is a list of attributes on which to group.

Each F_i is an aggregate function,

Each A_i is an attribute name.

Example

Relation r			O/P of $\text{sum}(c)(r)$
A	B	C	
		7	
		7	
		3	
		10	

(c) Outer Join

- An extension of the join operation that avoids loss of information.
- Computes the join and then adds tuples from one relation that do not match tuples in the other relation to the result of the join.
- **Uses Null Values**

(a) Null signifies that the value is unknown or does not exist.

(b) All comparisons involving null are false by definition.

Examples

Relation loan			Relation borrower	
loan-no	branch-name	amount	Customer-name	loan-no
L-170	Downtown	3000	Jones	L-170
L-230	Redwood	4000	Smith	L-230
L-260	Perridge	1700	Hayes	L-155

Inner Join ($\bowtie$)

Loan $\bowtie$ Borrower

loan-no	branch-name	amount	customer-name
L-170	Download	3000	Jones
L-230	Redwood	4000	Smith

Left outer join ($\Join$)

Loan $\Join$ Borrower

loan-no	branch-name	amount	customer-name
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-260	Perriridge	1700	Null

Right outer join ($\Join$)

Loan $\Join$ borrower

loan-no	branch-name	amount	customer-name
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-155	null	null	Hayes

Full outer join ($\Join$)

Loan $\Join$ borrower

loan-no	branch-name	amount	customer-name
L-170	Download	3000	Jones
L-230	Redwood	4000	Smith
L-260	Perriridge	1700	Null
L-155	null	null	Hayes

1.6.2 Modification of the Database

1.8.3.1 Deletion

In relational algebra, a deletion is expressed by

$$r \leftarrow r - E$$

where,

r is a relation and

E is a relational algebra query.

Example

Delete all of Smith's account records.

depositor $\leftarrow$ depositor - $\sigma_{\text{customer.name} = \text{"Smith"}}$ = (depositor)

Insertion

To insert data into a relation, we either a tuple to be inserted or write a query whose result is a set of tuples to be inserted.

Insertion is expressed by

$$R \leftarrow r \cup E$$

Where,

r is a relation and

E is a relational-algebra expression.

Example

To insert a fact that Smith has \$1200 in account A - 973 at the Perryridge branch.

$\text{account} \leftarrow \text{account} \cup \{(A-973, \text{"Perryridge"}, 1200)\}$

$\text{depositor} \leftarrow \text{depositor} \cup \{(\text{"Smith"}, A-973)\}$

Updation

To change a value in a tuple without changing all values in the tuples, the generalized projection operator is used.

$$r \leftarrow \pi_{F_1, F_2, \dots, F_n}(\sigma_r)$$

where each F_i is either the i^{th} attribute of r , if the i^{th} attribute is not updated, or, if the attribute is to be updated. F_i is an expression, involving only constants and the attributes of r , which gives the new value for the attribute.

If we want to select some tuples from r and to update only them, we can use the following expression; here, p denotes the selection condition that chooses which tuples to update:

$$R \leftarrow \pi_{F_1, F_2, \dots, F_n}(\sigma_p(r)) \cup (r - \sigma_p(r))$$

Example

To illustrate, the use of the update operation, suppose that interest payments are being made, and that all balances are to be increased by 5 percent.

$$\text{account} \leftarrow \pi_{\text{account-number}, \text{branch-name}, \text{balance} * 1.05}(\text{account})$$

Now suppose that accounts with balances over \$10,000 receive 6 percent interest, whereas all others receive 5 percent.

$$\text{account} \leftarrow \pi_{AN, BN, \text{balance} * \begin{cases} 1.06 & \text{if } \text{balance} > 10000 \\ 1.05 & \text{otherwise} \end{cases}}(\text{account})$$

$$U \leftarrow \pi_{AN, BN, \text{balance} * 1.06}(\sigma_{\text{balance} > 10000}(\text{account})) \cup \pi_{AN, BN, \text{balance} * 1.05}(\sigma_{\text{balance} \leq 10000}(\text{account}))$$

where the abbreviations AN and BN stand for account-number and branch-name, respectively.

SQL FUNDAMENTALS

Structured Query Language (SQL) is the standard common set used to communicate with the relational database management systems. All tasks related to relational data management—creating tables, querying the database for information, modifying the data in the database, deleting them, granting access to users, and so on — can be done using SQL.

Advantages of SQL

- ❖ SQL is a high level language that provides a greater degree of abstraction than procedural languages. It is so fashioned that the programmer can specify what data is needed but need not specify how to retrieve it. SQL is coded without embedded data-navigational instructions. This will be taken care of by the DBMS.
- ❖ SQL enables the end-users and systems personnel to deal with a number of database management systems where it is available. Increased acceptance and availability of SQL are also in its favor.
- ❖ Applications written in SQL can be easily ported across systems. Such porting could be required when the underlying Database Management System needs to be upgraded or changed.
- ❖ SQL as a language is independent of the way it is implemented internally. A query returns the same result regardless of whether optimizing has been done with indexes or not. This is because SQL specifies what is required and not how it should be done.
- ❖ The language while being simple and easy to learn can handle complex situations.

Parts of SQL

The SQL language has several parts:

- ❖ **Data-definition Language(DDL).** The SQL DDL provides commands for defining relation schemas, deleting relations and modifying relation schemas.
- ❖ **Interactive data-manipulation language (DML).** The SQL DML includes a query language based on both the relational algebra and the tuple relational calculus. It includes also commands to insert tuples into, delete tuples from, and modify tuples

in the database.

- ❖ **View definition.** The SQL DDL includes commands for defining views.
- ❖ **Transaction control.** SQL includes commands for specifying the beginning and ending of transactions.
- ❖ **Embedded SQL and Dynamic SQL.** Embedded and dynamic SQL define how SQL statements can be embedded within general-purpose programming languages, such as C, C++, Java, PL/I, COBOL, Pascal and Fortran.
- ❖ **Integrity.** The SQL DDL includes commands for specifying integrity constraints that the data stored in the database must satisfy. Updates that violate integrity constraints are disallowed.
- ❖ **Authorization.** The SQL DDL includes commands for specifying access rights to relations and views.

Domain Types of SQL

The SQL standard supports a variety of built-in domain types, including:

- ❖ **char(*n*):** A fixed-length character string with user-specified length *n*. The full form, **character** can be used instead.
- ❖ **varchar(*n*):** A variable-length character string with user-specified maximum length *n*. The full form, **character varying**, is equivalent.
- ❖ **int:** An integer (a finite subset of the integers that is machine dependent). The full form, **integer**, is equivalent.
- ❖ **smallint:** A small integer (a machine-dependent subset of the integer domain type).
- ❖ **numeric(*p,d*):** A fixed-point number with user-specified precision. The number consists of *p* digits (plus a sign), and *d* of the *p* digits are to the right of the decimal point. Thus, **numeric(3,1)** allows 44.5 to be stored exactly, but neither 444.5 or 0.32 can be stored exactly in a field of this type.
- ❖ **real, double precision:** Floating-point and double precision floating-point numbers with machine-dependent precision.

- ❖ **float(*n*)**: A floating-point number, with precision of at least *n* digits.
- ❖ **date**: a calendar date containing a (four-digit) year, month, and day of the month.
- ❖ **time**: the time of day, in hours, minutes and seconds. A variant, **time(*p*)**, can be used to specify the number of fractional digits for seconds (the default being 0). It is also possible to store time zone information along with the time.
- ❖ **timestamp**: A combination of **date** and **time**. A variant, **timestamp(*p*)**, can be used to specify the number of fractional digits for seconds (the default here being 6).

Date and time values can be specified like this:

date '2001-01-24'

time '09:30:00'

timestamp '2001-04-25 10:29:01.45'

Dates must be specified in the format year followed by month followed by day, as shown. The seconds field of **time** or **timestamp** can have a fractional part, as in the timestamp above. We can use an expression of the form **cast *e* as *t*** to convert a character string (or string valued expression) *e* to the type *t*, where *t* is one of **date**, **time**, or **timestamp**. The string must be in the appropriate format as illustrated at the beginning of this paragraph.

To extract individual fields of a **date** or **time** value *d*, we can use **extract** (field **from** *d*), where field can be one of **year**, **month**, **day**, **hour**, **minute** or **second**.

1.9.4. SQL LANGUAGE

Data Definition Language:

It is used to create a table, alter the structure of a table and also drop the table created.

Create Command

It is used to create a table.

Syntax

```
create table <table
name>
(columnname1
datatype(size),
columnname2
datatype(size)...);
```

Example

```
SQL>create table employee (ename varchar(10), eid number(5), address
```

vchar2(10), salary number(5), designation varchar2(10));

Alter Command

It is used to add a new column or modify existing column definitions.

Syntax

alter table <tablename> **add** (new columnname1 datatype(size),
new columnname2 datatype(size)...);

alter table <table name> **modify** (column definition);

Example

SQL>**alter table** employee **modify**(eid **number**(7));

SQL>**alter table** employee **add** (age **number**(2));

Drop Command

This command is used to delete a table.

Syntax

drop table <tablename>;

Example

SQL>**drop table** employee;

Notes: This command will delete the contents as well as structure.

Truncate Command

This is used to delete the records but retain the structure.

Syntax

truncate table <tablename>;

Example

SQL>**truncate table** employee;

To view the table structure

Syntax

desc <tablename>;

Example

SQL>**desc** employee;

Data Manipulation Language:

Insert Command

It is used to insert a new record in the database.

Syntax

insert into <tablename> **values** (a list of data values);

Example

SQL>**create table** employee (ename varchar2(10), eid number (5),
salary number(5));

SQL>**insert into** employee **values**('ABC',50,1000);

Update Command

Changes can be made by using update command.

Syntax

update <tablename> **set** field=value,..... **where**
<condition>;

Example

SQL>**update** employee **set** eid=100 **where** ename = 'ABC';

Delete Command

Rows can be deleted using delete command

Syntax

delete from <tablename> **where** <condition>;

Example

SQL>**delete from** employee **where** eid=100;

1.9.4.1 Data Control Language:

- ✓ Used to control privilege in Database.
- ✓ To perform any operation in the database, such as for creating tables, sequences or views we need privileges.

Grant Command

Gives user access privileges to database.

Syntax

Grant Select/insert/delete/update/alter/all privileges on tablename to authenticate;

Example

Grant select, update on student to vikram;

Revoke Command

Take back permissions from user.

Syntax

Revoke Select/insert/delete/update/alter/all privileges on tablename from authenticate;

Example

Revoke select, update on student from vikram;

Transaction Control Language:

- ✓ Used to manage transactions in database
- ✓ To manage the changes made by DML statements.
- ✓ Allows statements to be grouped together into logical transactions.

Commit command

Commit command is used to permanently save any transaction into database.

Syntax Commit;

Rollback Command

Restores the database to last committed state. It is also use with savepoint command to

jump to a savepoint in a transaction.

Syntax

rollback to savepoint_name;

Example

rollback to Temp;

Savepoint Command

Used to temporarily save a transaction so that you can rollback to that point whenever necessary.

Syntax

Savepoint savepoint_name;

Example

Savepoint Temp

1.10. ADANCED SQL FEATURES

The structure of an SQL expression consists of three clauses : select, from and where

The **select clause** corresponds to the projection operation of the relational algebra. It is used to list the attributes desired in the result of a query.

The **from clause** corresponds to the Cartesian product operation of the relational algebra. It lists the relations to be scanned in the evaluation of the expression.

The **where clause** corresponds to the selection predicate of the relational algebra. It consists of a predicate involving attributes of the relations that appear in the from clause.

A SQL has the form

select A₁,A₂,A_n

from r₁,r₂, r_m

where p

A_i - an attribute

r_i - relation

p - predicate

The query is equivalent to the relational algebra expression.

$$\sigma_{A_1, A_2, \dots, A_n} (\sigma_p (r_1 \times r_2 \times \dots \times r_m))$$

If the where clause is omitted, the predicate p is true.

Tuple Variables

Tuple variables are defined in the from clause via the use of the **as clause**.

For example, find the customer names and their loan numbers for all customers having a loan at some branch in the banking database.

```
SQL>select B.customer_name, B.loan_no, L.amount
      from borrower as B, loan as L where
      L.loan_no = B.loan_no;
```

String Operations

SQL includes a string matching operator for comparisons on character strings. Patterns are described using 2 special characters.

- ☐ **Percent (%):** The % character, matches any substring.
- ☐ **Underscore (-):** The -character matches any character.

Example: Find the names of customers where the 1st 2 characters are 'Ba'.

```
SQL>select customer_name from customer where
      customer_name like 'Ba%';
```

Find the names of customer where the 2nd character is 'n' or 'a'.

```
SQL>select customer_name from customer where
      customer_name like '_n%' or '_a%';
```

- ☐ Patterns are case sensitive, i.e., upper case characters do not match lowercase characters, and vice versa.

- SQL supports a variety of string operations such as
 - (a) Concatenation using '||' or concat ()
 - (b) Converting the string into upper or lower case upper or lower ()
 - (c) Find string length (strlen ()),
extracting substring (substr ()), etc.

Order by Clause

The order by clause causes the tuples in the result of a query to appear in sorted order.

```
SQL>select *  
      from employee  
      order by salary;  
SQL>select *  
      from employee  
      order by salary desc, eid asc;
```

Aggregate Functions

Aggregate functions are functions that take a collection of values as input and return a single value as output.

SQL offers 5 built in aggregate

functions: avg -	
average value	
min -	minimum value
max -	maximum value
sum -	sum of values
count -	number of values.

Examples

1. Find the average account balance at the perryridge

branch: SQL>**select avg (balance) from**
account where

branch-name = "Perryridge";

2. Find the number of tuples in the customer

```
relation SQL>select count (*) from  
customer;
```

Aggregate functions with group by clause

Group by clause is used to group the rows based on certain criteria. Group by is used in conjunction with aggregate functions like sum, avg, min, max, count, etc.

Example: Find the average account balance at each branch.

```
SQL>select branch_name, avg (balance) from account  
group by branch_name;
```

Aggregate functions with having clause

Find the name of all branches where the average account balance is more than \$2,000.

```
SQL>select branch_name, avg (balance) from account  
group by branch_name having avg (balance) > 2000;
```

Null Values

- ☐ SQL allows the use of null values to indicate absence of information about the value of an attribute.

- ☐ Null signifies an unknown value or that a value does not exist.

Example:

- ☐ The predicate is null can be used to check for null values.

Find all loan numbers which appear in the loan relation with null values for amount.

```
SQL>select loan_no from loan where amount is null;
```

- ☐ The result of any arithmetic expression (involving, for example +, -, *, or /) is null if any of the input values is null.

Example: 5 + null returns null.

Consider the unknown (null) value used in boolean expressions as

1. OR - (unknown or true) = true, (unknown or false) = unknown (unknown or unknown) = unknown.
2. AND - (true and unknown) = unknown, (false and unknown) = false, (unknown and unknown) = unknown.
3. NOT - (not unknown) = unknown.

- ☐ For example, find total of all loan amounts. SQL>**select sum (amount)**
from loan;

Above statement ignores all null amounts. The result is null if there is no non- null amount.

- ☐ All aggregate operations except count (*) ignore tuples with null values on the aggregated attributes.

Nested Subqueries

- ☐ SQL provides a mechanism for the nesting of subqueries. A subquery is a select-from-where expression that is nested within another query.
- ☐ A common use of subquery is to perform tests for set membership, set comparisons and set cardinality.

Set Membership

SQL uses in and not in constructs to set membership tests.

IN

The IN connective tests for set membership, where the set is a collection of values produced by a select clause.

Example: Find all customers who have both an account and a loan at the bank.

SQL>**select distinct** customer-name **from** depositor

where customer-name **in** (**select** customer-name **from** borrower);

NOT IN

The not in connective tests for the absence of set membership.

Example: Find all customers who have a loan at the bank but do not have an account at the bank.

SQL>**select distinct** customer-name **from** borrower

where customer-name **not in** (**select** customer name **from** depositor);

2. Set Comparison

Nested queries are used to compare sets. SQL uses various comparison operators such as <, >, <=, >=, <>, any, all, and, some, etc. to compare sets.

Example: Find the names of all branches that have assets greater than those of atleast one branch located in “Chennai”.

SQL>**select distinct** T. branchname **from** branch **as** T,

branch **as** S **where** T. assets > S. assets

and S. branch-city = “Chennai”;

Same query using > **some** clause.

SQL>**select** branch-name **from** branch **where**

assets > **some** (**select** assets **from** branch

where branch-city = “Chennai”);

SQL also allows < **some**, <= **some**, >= **some**, = **some**, and <> **some** comparisons, = **some** is identical to in, and <> **some** is identical to not in. The keyword **any** is synonym to **some** in SQL.

SQL also allows < **all**, > **all**, <= **all**, >= **all**, = **all**, and <> **all** comparisons <> **all** is identical to **not in**.

Example: Find the names all branches that have an-assets value greater than that of each branch in “Chennai”.

```
SQL>select branch-name from branch where  
assets > all (select assets from branch where  
branch-city = “Chennai”);
```

3. Test for Empty Relations

- ☐ SQL includes a feature for testing whether a subquery has any tuples in its result.
- ☐ The **exists** construct returns the value true if the argument subquery is non-empty.

Example: Find all customers who have both an account and a loan at the bank.

```
SQL>select customer-name from borrower where exists  
(select * from depositor where depositor.customer-name  
= borrower.customer-name);
```

Similar to **exists** we can use **not exists** also. Find all customers who have an account at all branches located in “Chennai”.

```
SQL>select distinct S. customer-name from depositor as S  
where not exists ((select branch-name from branch  
where branch-city = “Chennai”) except  
(select R. branch_name from depositor as T, account as R  
where T. account_no = R. account_no and  
S. customer-name = T. customer-name));
```

4. Test for absence of duplicate tuples

The **unique** construct tests whether a subquery has any duplicate tuples in its result.

Example: Find all customers who have at most one account at the “Chennai branch”.

```
SQL>select T. customer_name from depositor as T  
  
      where unique (select R. customer-name  
  
      from account, depositor as R  
  
      where T. customer_name = R. customer-name  
  
      and R. account-no = account.account_no and  
  
      account.branch_name = “Chennai”);
```

not unique construct is used for test the existence of duplicate tuples in the same manner.

Complex Queries

Complex queries are often hard or impossible to write as a single SQL block.

There are two ways for composing multiple SQL blocks to express a complex query.

(1) derived relations (2) with clause.

1. Derived relations

SQL allows a subquery expression to be used in the **from** clause. If we use such an expression, then we must give the result relation a name, and we can rename the attributes. For renaming **as** clause is used. For example, find the average account balance of those branches where the average account balance is greater than \$ 2000.

```
SQL>select branch-name, avg-balance from (select branch-name,  
  
      avg (balance) from account group by branch-name)  
  
      as branch-avg (branch-name, avg-balance) where  
  
      avg-balance > 2000;
```

Here subquery result is named branch-avg with the attributes branch-name and avg-balance.

2. With Clause

The with clause provides a way of defining a temporary view, whose definition available only to the query in which the **with** clause occurs.

Consider the following query, which selects **accounts with the maximum balance**. If there are many accounts with the same maximum balance, all of them are selected.

```
with max_balance (value) as  
  
select max (balance)  
  
from account  
  
select account_no from account,  
  
max_balance where account.balance =  
  
max_balance.value;
```

Views

- ☐ A view is an object that gives the user a logical view of data from an underlying table or tables (relation or relations).
- ☐ It is not desirable for all users to see the entire logical model.
- ☐ Security considerations may require that certain data be hidden from users.
 - Any relation that is not part of the logical model, but is made visible to a user as a **virtual relation**, is called as **view**.

Views may be created for the following reasons:

- (d) to provide data security
- (e) query simplicity
- (f) structural simplicity (because view contains only limited number of columns and rows).

(1) Creation of

Views Syntax

```
create view view_name as < query expression >;
```

Example: Create a view customer_details from customer relation with customer name and customer-id.

```
create view customer_details as  
  
select customer_name, customer_id from customer;
```

(2) Assigning Names to Columns

We can assign names for the various columns in the view. This may be entirely different from what has been used in the main relation.

For example,

```
SQL>create view customer_details (cust_name, customer_no)  
  
as select customer_name, customer_id from customer;
```

(3) *Selecting data from a view*

```
SQL>select * from customer_details;
```

(4) *Updation of a view*

- ☐ Views can also be used for data manipulation i.e., the user can perform insert, update, and the delete operations on the view.
- ☐ The views on which data manipulation can be done are called **Updatable views**, the views that do not allow data manipulation are called **Readonly views**.
- ☐ When you give a view name in the update, insert, or delete statement, the modification to the data will be passed to the underlying (main) relation.

For the view to be updatable, it should meet following criteria:

1. The view must be created on a single table.
2. Primary key column of the table should be included in the view.
3. Aggregate functions cannot be used in the select statement.
4. The select statement used for creating a view should not include distinct, group by or having clause.
5. Select statement used for creating a view should not include subqueries.
6. It must not use constant, strings or value expression like total/6.

(5) Destroying a view

A view can be dropped by using the drop view command.

Syntax

drop view view_name;

Example

SQL>**drop view** customer_details;

1.10.2 Joins

A join is a query using which we can query data more than one table. Joins are the basic of multi-table query processing in SQL. A join is a query that extracts corresponding rows from two or more tables, views or snapshots. If the two tables used in the join have the same column name, then the column names should be prefixed with table name followed by a period.

SELECT statement of a multi-table query must contain a filter condition that specify the column match. The **where** clause is used to specify the selection condition and the join condition. In the where clause the logical operators can also be used.

Types of joins

Joins are classified into four types namely:

1. Inner Join
2. Outer Join
3. Natural Join

1. Inner Join

Inner join returns the matching rows from the tables that are being joined.

Consider following two relations:

- i) Student(sname,place)
- ii) Student_marks(sname,dept,mark)

Student	sname	place
	Prajan	Chennai
	Anand	Kolkata
Prajan	Kumar	Delhi 700
Anand	Ravi	Mumbai 650
		680
		IT 600

Example 1

SQL>**select** Student.sname,Student_marks,mark **from** Student **inner join** Student_marks
on Student.sname=Student_marks.sname;

The output of the above query is

sname	mark
Prajan	700
Anand	650
Ravi	600

Example 2:

```
SQL>select * from Student inner join Student_marks on  
Student.sname=Student_marks.sname;
```

The result of the above query is

sname	place	sname	dept	mark
Prajan	Chennai	Prajan	CSE	700
Anand	Kolkata	Anand	IT	650
Ravi	Mumbai	Ravi	IT	600

For example 2 the result consists of the attributes of the left-hand-side relation followed by the attributes of the right-hand-side relation. Thus, the sname attribute appears twice in result, first is from student table and second is from student_marks table.

2. Outer Join

When tables are joined using inner join, rows which contain matching values in the join predicate are returned. Sometimes you may want both matching and non-matching rows returned for the tables that are being joined. This kind of operation is known as an outer join.

An outer join is an extended form of the inner join. In this, the rows in one table having no matching rows in the other table will also appear in the result table with nulls.

Types of Outer Join

The Outer Join can be any one of the

- following: o Left outer
- o Right outer

1.Left outer join

The left outer join returns matching rows from the tables being joined and also non-matching rows from the left table in the result and places null values in the attributes that come from the right table.

Example 3

```
SQL>select Student.sname, Student_marks.mark from Student left outer join  
Student_marks on Student.sname = Student_marks.sname;
```

The result of above query is

sname	mark
Prajan	700
Anand	650
Ravi	600
Kumar	null

Left outer join operation is computed as follows:

First compute the result of inner join as before. Then, for every tuple 't' in the left hand side relation, Student that does not match any tuple in the right-hand side relation Student_marks in the inner join, add a tuple 'r' to the result of the join: The attributes of tuple 'r' that are derived from the left-hand-side relation are filled with from tuple 't', remaining attributes of 'r' are filled with null values as shown in example 3.

2.Right outer join

The right outer join operation returns matching rows from the tables being joined, and also non-matching rows from the right table in the table in the result and places null values in the attributes that comes from the left table.

Example 4

```
SQL>select Student.sname,Student.place,Student_marks.mark from Student  
right outer join Student_marks on Student.sname =
```

Student_marks.sname; The result of above query is

sname	place	mark
Prajan	Chennai	700
Anand	Kolkota	650
Ravi	Mumbai	600
Vasu	null	680

EMBEDDED SQL

SQL provides a powerful declarative query language. However, access to a database from a general-purpose programming language is required because,

- SQL is not as powerful as a general-purpose programming language. There are queries that cannot be expressed in SQL, but can be programmed in C, Fortran, Pascal, Cobol, etc.
- Non declarative actions -- such as printing a report, interacting with a user, or sending the result to a GUI -- cannot be done from within SQL.

The SQL standard defines embedding of SQL as *embedded SQL* and the language in which SQL queries are embedded is referred as *host language*.

An embedded SQL statement is distinguished from the host language statements by enclosing it between **EXEC SQL** or **EXEC SQL BEGIN** and a matching **END-EXEC** or **EXEC SQL END** (or semicolon)

- Syntax may vary with language
- *Shared variables* (used in both languages) usually prefixed with a colon (:) in SQL

Embedded SQL in C Program Examples

Example 1

/* Variable Declaration in Language C */

- Variables inside **DECLARE** are shared and can appear (while prefixed by a colon) in SQL statements
- **SQLCODE** is used to communicate errors/exceptions between the database and the program

```

int loop;

EXEC SQL BEGIN DECLARE SECTION;

    varchar dname[16], fname[16], ...;

    char ssn[10], bdate[11], ...;

    int dno, dnumber, SQLCODE, ...;

EXEC SQL END DECLARE SECTION;

```

Example 2

/* Conditional and Looping Statements in Language C */

```

loop = 1;

while (loop) {

    prompt ("Enter SSN: ", ssn);

    EXEC SQL

        select FNAME, LNAME, ADDRESS, SALARY

        into :fname, :lname, :address, :salary

        from EMPLOYEE where SSN == :ssn;

    if (SQLCODE == 0) printf(fname, ...);

    else printf("SSN does not exist: ", ssn);

    prompt("More SSN? (1=yes, 0=no): ",

loop); END-EXEC

}

```

Example 3

/*Embedded SQL in C Programming using Cursor */

Embedded C program to do the following:

1. Starting with a station name (Chennai, in this example), look up the station ID.

2. Print the station ID.
3. List all rows for that station ID.

- shows single-row select and use of cursor
- Note that all C-language variables used in SQL statements are declared in the DECLARE SECTION.
- Note that all SQL statements begin with the syntax EXEC SQL and terminate with a semicolon.

```
#include<stdio.h>
#include<string.h>
EXEC SQL BEGIN DECLARE SECTION;
    long station_id;
    long mon;
    float temp;
    float rain;
    char city_name[21];
    long SQLCODE;
EXEC SQL END DECLARE SECTION;
main()
{
/* the CONNECT statement, if needed, goes here
*/ strcpy(city_name,"Chennai");
EXEC SQL SELECT ID INTO
    :station_id FROM STATION
    WHERE CITY = :city_name;
if (SQLCODE == 100)
{
    printf("There is no station for city %s\n",city_name);
    exit(0);
}

printf("For the city %s, Station ID is %ld\n",city_name,station_id);
printf("And here is the weather data:\n");
```

```

EXEC SQL DECLARE XYZ CURSOR FOR
    SELECT MONTH, TEMP_F, RAIN_I
    FROM STATS
    WHERE ID =
        :station_id ORDER BY
        MONTH;
EXEC SQL OPEN XYZ;
while (SQLCODE != 100) {
    EXEC SQL FETCH XYZ INTO :mon, :temp,
        :rain; if (SQLCODE == 100)
        printf("end of list\n");
    else
        printf("month = %ld, temperature = %f, rainfall = %f\n",mon,temp,rain);
}

EXEC SQL CLOSE XYZ;
exit(0);
}

```

Output

```

For the city Chennai, Station ID is
44 And here is the weather data:
month = 1, temperature = 27.299999, rainfall = 0.180000
month = 7, temperature = 74.800003, rainfall = 2.110000
end of list

```

DYNAMIC SQL

Unlike static SQL, which remains the same in each execution, dynamic SQL enables you to build SQL statements as character strings at runtime. The strings contain the text of a SQL statement or PL/SQL block and can also contain placeholders for bind arguments.

You can create general purpose, flexible applications with dynamic SQL because the full text of a SQL statement may be unknown at compilation. You can use dynamic SQL in several different development environments, including PL/SQL, Pro*C/C++, and Java.

For an example of an application that uses dynamic SQL, suppose that a reporting

application in a data warehouse environment does not know a table name until runtime. These tables are named according to the starting month and year of the quarter, for example, inv_01_2003, inv_04_2003, inv_07_2003, inv_10_2003, inv_01_2004, and so on. You can use dynamic SQL in your reporting application to specify the table name at runtime.

In a different example, suppose that you want to run a complex query with a user-selectable sort order. Instead of coding the query twice with a different ORDER BY clause in each query, you can construct the query dynamically to include a specified ORDER BY clause.

Programming with Dynamic SQL

For the sake of consistency, this chapter discusses dynamic SQL mainly from the perspective of PL/SQL. To process most dynamic SQL statements, you use the EXECUTE IMMEDIATE statement. To process a multi-row query in a PL/SQL procedure, you use the OPEN-FOR, FETCH, and CLOSE statements.

Oracle Database enables you to implement dynamic SQL in a PL/SQL application in the following ways:

- Using native dynamic SQL, which involves placing dynamic SQL statements directly into PL/SQL blocks
- Calling procedures in the DBMS_SQL package

Although this chapter discusses PL/SQL support for dynamic SQL, you can call dynamic SQL from other languages:

- If you use C/C++, you can call dynamic SQL with the Oracle Call Interface (OCI), or you can use the Pro*C/C++ precompiler to add dynamic SQL extensions to your C code.
- If you use COBOL, you can use the Pro*COBOL precompiler to add dynamic SQL extensions to your COBOL code.
- If you use Java, you can develop applications that use dynamic SQL with JDBC.

Why Use Dynamic SQL?

Dynamic SQL and static SQL both have advantages and disadvantages. The full text of

static SQL statements is known at compilation, which provides the following benefits:

- Successful compilation verifies that the SQL statements reference valid database objects and that the necessary privileges are in place to access the objects.
- Performance of static SQL is generally better than dynamic SQL.

Despite these advantages, static SQL has limitations that can be overcome with dynamic SQL, as in the following cases:

- We do not know the full text of the SQL statements that must be executed in a PL/SQL procedure. These SQL statements may depend on user input or on processing work performed by the program.
- We want to execute DDL statements and other SQL statements that are not supported in purely static SQL programs.
- We want to write a program that can handle changes in data definitions without the need to recompile. Dynamic SQL is more flexible than static SQL because it enables you to write reusable code that can be adapted for different environments.

Executing DDL Statements in PL/SQL

Only dynamic SQL can execute the following types of statements within PL/SQL program units:

- Data definition language (DDL) statements such as CREATE, DROP etc.,
- The TABLE clause in the SELECT statement

The following native dynamic SQL example uses a SELECT statement with the TABLE clause.

Example 1 Using SELECT . . . TABLE in Dynamic SQL

```
-- Create an object t_emp and a datatype t_emplist as a table of type t_emp
CREATE TYPE t_emp AS OBJECT (id NUMBER, name VARCHAR2(20))
/
CREATE TYPE t_emplist AS TABLE OF t_emp
```

```

/

-- Create a table with a nested table of type t_emplist
CREATE TABLE dept_new (id NUMBER, emps t_emplist)
NESTED TABLE emps STORE AS emp_table;

-- Populate the dept_new table with data
INSERT INTO dept_new VALUES
(
    10,
    t_emplist
    (
        t_emp(1, 'Pradeep'),
        t_emp(2, 'Kumar')
    )
);

-- Write a PL/SQL block that queries table dept_new and nested table emps
-- SELECT ... FROM ... TABLE is not allowed in static SQL in PL/SQL
DECLARE
    v_deptid NUMBER;
    v_ename VARCHAR2(20);
BEGIN
    EXECUTE IMMEDIATE 'SELECT d.id, e.name
                        FROM dept_new d, TABLE(d.emps) e
                        WHERE e.id = 1'
    INTO v_deptid, v_ename;
END;

```